

watchlist-monitor

Architecture & Code Guide

A complete walkthrough of every module, data flow, API, frontend and deployment

This document explains how watchlist-monitor is built — what each file does, how data flows from a market close to your email/screen, and how all the pieces fit together. Aimed at developers and power users who want to understand, extend or debug the system.

Chapter 1: Project Overview

watchlist-monitor is a **personal stock alert and screener tool** built on Python. It fetches real market data, computes technical indicators, evaluates configurable trigger conditions, surfaces research candidates, and delivers an HTML report by email — all running free on GitHub Actions or locally via a FastAPI web UI.

Core design principles:

1. **Alert tool, not prediction tool.** Every output describes current technical conditions. Nothing forecasts prices.
2. **No code changes needed for normal use.** All user configuration lives in config.yaml or the Settings tab.
3. **One bad ticker never aborts the run.** Every fetch and evaluate call is wrapped; errors are surfaced per-ticker, not globally.
4. **Pure functions at the core.** indicators.py, portfolio.py, assessment.py, screener.py, themes.py — all side-effect-free and fully unit-tested.

Technology Stack

Layer	Technology	Purpose
Data fetch	yfinance (0.2.x)	Daily OHLC price history from Yahoo Finance
Indicators	numpy + pandas	RSI, SMA, daily %, MA crossover
Persistence	SQLite (stdlib)	Watchlist, portfolio, settings for the UI
Backend API	FastAPI + uvicorn	REST JSON API serving the web frontend
Frontend	Vanilla JS + Chart.js	SPA — no build step, no npm
Scheduler	APScheduler	In-app daily email job
Email	smtplib (stdlib)	SMTP send or stdout dry-run
AI notes	InvestorMate + OpenAI	Optional context notes via Investor.ask()
CI/CD	GitHub Actions	Free weekday cron, no server needed
Config	PyYAML	Human-editable config.yaml
Tests	pytest	85 tests covering all pure modules + API

Chapter 2: File Map

File / Directory	Role
config.yaml	User-editable watchlist, thresholds, AI settings. Seeded into SQLite on first UI launch.
monitor/__init__.py	Empty — marks monitor/ as a Python package.
monitor/indicators.py	Pure functions: wilder_rsi(), sma(), daily_pct_change(), ma_crossover(). Zero I/O.
monitor/triggers.py	evaluate(ticker, hist, cfg) — runs all indicators, returns snapshot + alert strings.
monitor/assessment.py	assess(result, position, cfg) — deterministic label from indicator votes. Zero I/O.
monitor/core.py	fetch_history(), fetch_many() (bulk), scan() — the shared pipeline. 15-min cache.
monitor/portfolio.py	compute_position(), summarize() — pure P&L math. Zero I/O.
monitor/screener.py	build_ideas() — ranks tickers by signal count. Used by Daily Ideas + Market Scan.
monitor/themes.py	THEME_TICKERS map + expand_interests() — theme name -> ticker list.
monitor/universe.py	Index constituent lists: Dow30, Nasdaq100, NIFTY50, SENSEX30.
monitor/store.py	SQLite CRUD for holdings, interests, settings. build_cfg() reconstructs config dict.
monitor/ai_notes.py	get_ai_notes() — calls InvestorMate Investor.ask(). Skips gracefully if not configured.
monitor/emailer.py	build_html() + send_or_print() — HTML report, SMTP send or stdout dry-run.
monitor/run.py	CLI entry point: python -m monitor.run. Thin wrapper around core.scan +emailer.
webapp/__init__.py	Empty — marks webapp/ as a package.
webapp/main.py	FastAPI app: all /api/* routes, APScheduler setup, static file mount.
webapp/static/index.html	Single-page app shell. Six tabs. Chart.js via CDN.
webapp/static/app.js	All frontend logic: tab routing, API calls, Chart.js rendering.
webapp/static/styles.css	Dark-theme CSS. No framework.
.github/workflows/daily.yml	Weekday 21:30 UTC cron + manual dispatch. Runs monitor/run.py.
tests/	85 pytest tests: indicators, triggers, portfolio, assessment, screener, themes, store, API.
watchlist.db	SQLite database (git-ignored). Created on first uvicorn startup.

Chapter 3: Data Flow — From Market Close to Screen/Email

The pipeline is identical whether triggered by the CLI, GitHub Actions, the web UI dashboard, or the APScheduler daily job:

Step	Where	What happens
1. Load config	<code>screen.build_cfg()</code> or <code>config.yaml</code>	Tickers, thresholds, AI flag assembled into a <code>cfg</code> dict
2. Fetch history	<code>store.fetch_history()</code> / <code>fetch_many()</code>	13 months of daily OHLC via <code>yfinance</code> . 15-min cache.
3. Compute indicators	<code>indicators.py</code> (called by <code>triggers.py</code>)	RSI, SMA50, SMA200, daily %, MA crossover
4. Evaluate triggers	<code>triggers.evaluate()</code>	Compare indicators to thresholds; build alert strings
5. Assess	<code>assessment.py assess()</code>	Vote-based label: strong / weak / stretched / neutral
6. Portfolio P&L	<code>Portfolio.py</code> (UI/API only)	Market value and unrealized P&L per holding
7. AI notes (optional)	<code>options.get_ai_notes()</code>	<code>Investor.ask()</code> per ticker if enabled + key present
8. Build report	<code>reporter.build_html()</code>	HTML table with all ticker rows, alerts, AI notes
9. Deliver	<code>emailer.send_or_print()</code>	SMTP if all env vars set; stdout dry-run otherwise

The Screener path (Daily Ideas / Market Scan) inserts between steps 5 and 8:

`screener.build_ideas(results, cfg)` groups tickers by which signals fired (oversold, overbought, golden cross, death cross, big movers) and ranks them by signal count + daily move magnitude. `group_limit` caps per-group items for the large index scans.

Chapter 4: Module Deep-Dives

monitor/indicators.py — Pure Indicator Math

Function / method	What it does
<code>wilder_rsi(prices, period=14)</code>	Returns a pd.Series of RSI values. First 'period' values are NaN (insufficient data). Uses explicit Wilder smoothing loop (not ewm) for correctness. Handles flat prices (both avg_g=0 and avg_l=0) by returning NaN rather than 100.
<code>sma(prices, period)</code>	Rolling mean. Returns NaN for the first period-1 values. One line: <code>prices.rolling(period).mean()</code> .
<code>daily_pct_change(prices)</code>	<code>pct_change() × 100</code> . First value is always NaN.
<code>ma_crossover(prices, short_period, long_period)</code>	Drops NaN from both SMA series, aligns them, checks the last two bars for a position change. Returns 'golden', 'death', or None.

monitor/triggers.py — Trigger Evaluation

Function / method	What it does
<code>evaluate(ticker, hist, cfg)</code>	Entry point for a single ticker. Calls all four indicator functions, compares to thresholds from <code>cfg['triggers']</code> , builds a list of human-readable alert strings, and returns a result dict: {ticker, price, change_pct, rsi, ma_cross, alerts, error}. Any exception sets error= and returns gracefully.

monitor/assessment.py — Signal Assessment

Function / method	What it does
<code>assess(result, position=None, cfg=None)</code>	Pure function. Reads rsi, ma_cross, change_pct from result. Assigns bull/bear votes. Label = strong (bull only), weak (bear only), stretched (both), neutral (neither). If a portfolio position is passed, adds P&L; context. Never emits buy/sell/hold verbs — tested explicitly in <code>test_assessment.py</code> .

monitor/core.py — Shared Scan Pipeline

Function / method	What it does
<code>fetch_history(ticker, use_cache=True)</code>	Single-ticker fetch via <code>yf.Ticker.history()</code> . Stores result in <code>_HIST_CACHE</code> with a 15-minute TTL. Prevents re-fetching the same ticker within a session.

Function / method	What it does
<code>fetch_many(tickers, refresh=False)</code>	Bulk fetch via <code>yf.download(group_by='ticker', threads=True)</code> . Returns a dict of DataFrames. Used by <code>market_scan</code> for 30–100 ticker batches. Honours the same cache.
<code>scan(cfg, tickers=None, histories=None, refresh=False, progress=None)</code>	Loops tickers, fetches (or uses pre-fetched histories dict), calls <code>evaluate()</code> . Never raises — errors are captured per-ticker. Optional progress callback for CLI logging.

monitor/store.py — SQLite Persistence

Function / method	What it does
<code>init_db()</code>	Creates tables (holdings, interests, settings) if absent. Calls <code>_seed_if_empty()</code> which imports <code>config.yaml</code> on first run.
<code>build_cfg()</code>	Reconstructs the <code>cfg</code> dict that <code>core.scan</code> + <code>ai_notes</code> expect: tickers from <code>watchlist_tickers()</code> (interests expanded via themes + holdings), triggers and <code>ai_analysis</code> from settings table.
<code>update_settings(patch)</code>	Shallow-merges top-level keys; nested dicts (triggers, <code>ai_analysis</code>) merge one level. Stored as JSON per key in the settings table.

monitor/screener.py — Screener

Function / method	What it does
<code>build_ideas(results, cfg, ai_notes={}, top_n=5, group_limit=None)</code>	Iterates results, assigns each ticker to one or more signal groups (oversold, overbought, golden, death, movers). Ranks all flagged tickers by (<code>signal_count</code> DESC, <code> change_pct </code> DESC). Returns <code>{top: [...], groups: [...], counts: {...}}</code> . <code>group_limit</code> caps per-group items for large scans (15 per group for index scans).

Chapter 5: The Web Application

Start with: `uvicorn webapp.main:app --host 127.0.0.1 --port 8000`

5.1 FastAPI Routes

Method + Path	Purpose
GET /api/dashboard	Scan all interests+holdings, enrich with assessment+AI notes. ?refresh=1 bypasses cache.
GET /api/ideas	Daily Ideas screener for user's own universe.
GET /api/universe	Returns index metadata (name, currency symbol, ticker count).
GET /api/market-scan?index=X	Bulk-fetch index constituents, run screener, return flagged names (top 8, groups capped at 8).
GET /api/ticker/{symbol}	Deep-dive: snapshot, assessment, 90-day price/SMA/RSI chart series, AI note, SMA relationship.
GET+POST /api/holdings	List or create holdings.
PUT+DELETE /api/holdings/{id}	Update or remove a holding.
GET+POST /api/interests	List (with available themes) or add an interest.
DELETE /api/interests/{id}	Remove an interest.
GET+PUT /api/settings	Read or update settings (triggers, ai_analysis, schedule, always_send_summary).
POST /api/email/preview	Returns HTML string for preview in an iframe.
POST /api/email/send	Runs the daily report job (SMTP or stdout dry-run).
POST /api/export-config	Writes current store state back to config.yaml for the Actions cron.
GET /	Serves webapp/static/index.html.
/* (static)	Serves CSS, JS, any other static assets.

5.2 In-App Scheduler (APScheduler)

A BackgroundScheduler starts on app startup. The Settings tab lets you toggle it on/off and set hour/minute. When enabled, a cron job runs weekdays at the configured time, calling the same pipeline as the CLI: scan → assess → build_html → send_or_print. Reschedules itself on every PUT /api/settings.

5.3 Frontend — Six Tabs

Tab	What it shows / does
Dashboard	Sortable table of all interests+holdings. Price, daily%, RSI, MA cross, alerts, assessment, AI note. Refreshes on demand.
Daily Ideas	Research candidates from your own universe. Top-ranked + by-signal groups. Refreshes on demand.
Market Scan	Pick an index (Dow30/Nasdaq100/NIFTY50/SENSEX30) and scan. Currency-aware (\$/Rs). Top 8 + groups.
Ticker Inspector	Type any ticker (US or Indian). Shows metric cards, 90-day price+SMA chart, RSI chart, alerts, assessment.

Tab	What it shows / does
Portfolio	Add/edit/delete holdings. Live market value + unrealized P&L per holding and in total.
Interests	Manage individual tickers and themes. Available themes dropdown. Chips list with delete.
Settings	Trigger thresholds, AI toggle, scheduler config, email preview/send, Export to config.yaml.

5.4 Chart.js Integration (Ticker Inspector)

Two Chart.js line charts are rendered per ticker inspection:

Price chart: 90 sessions of closing price (blue) + SMA50 (gold dashed) + SMA200 (red dashed). The SMA series have NaN values for the first period-1 bars, passed as null to Chart.js.

RSI chart: 90 sessions of RSI(14) (green) + static 70-line (red dashed) + static 30-line (amber dashed). Y-axis clamped 0–100.

Both charts are destroyed and recreated on each new inspection to prevent canvas reuse errors.

Chapter 6: GitHub Actions Deployment

File: `.github/workflows/daily.yml`

Property	Value
Trigger	Weekday cron at 21:30 UTC (~4:30 PM ET) + manual workflow_dispatch
Runner	ubuntu-latest (free GitHub Actions)
Python	3.11 with pip cache
Secrets needed	SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, MAIL_TO (all required for email); OPENAI_
Dry-run	If any SMTP secret is missing, the report prints to the Actions log instead of emailing
Config source	Reads config.yaml from the repo. Use 'Export to config.yaml' in Settings then commit to sync.

Setting up GitHub Secrets:

Repository → Settings → Secrets and variables → Actions → New repository secret

Add each of: SMTP_HOST, SMTP_PORT (587), SMTP_USER, SMTP_PASS (Gmail app password), MAIL_TO

For Gmail: enable 2FA → myaccount.google.com/apppasswords → generate a 16-char app password

Chapter 7: Test Suite (85 tests)

Run with: `python -m pytest tests/ -v`

Test file	What it covers
test_indicators.py	19 tests: SMA (4), Wilder RSI (6), daily% (4), MA crossover (5). Pure math, deterministic data.
test_triggers.py	17 tests: snapshot correctness, error cases, all four alert types (move, RSI, crossover).
test_portfolio.py	7 tests: P&L gain/loss/zero-cost, summarize() with missing prices.
test_assessment.py	8 tests: each label, position context, and a directive-language assertion (no buy/sell/hold).
test_screener.py	7 tests: grouping, sorting, ranking, error exclusion, directive-language assertion.
test_themes.py	4 tests: theme expansion, de-duplication, unknown theme, suffix correctness.
test_store.py	12 tests: holdings CRUD, interests deduplication, settings deep merge, build_cfg() assembly.
test_api.py	11 tests: full FastAPI TestClient tests for interests, holdings, portfolio P&L, dashboard assessment.
test_universe.py	6 tests: index presence, India .NS/.BO suffixes, US no-suffix, currency symbols, count consistency.

Key testing patterns:

- Pure modules tested with synthetic pandas Series — no network calls.
- FastAPI tests use TestClient with monkeypatched fetch_history / fetch_many — fully offline.
- Each test DB uses tmp_path (pytest fixture) — no shared state between tests.
- Directive-language tests: assert 'buy'/'sell'/'hold'/'price target'/'will rise'/'will fall' never appear in any assessment or screener output.

Chapter 8: How to Extend the Project

8.1 Adding a new indicator

1. Add a pure function to **monitor/indicators.py** following the `pd.Series → pd.Series` pattern.
2. Add unit tests to **tests/test_indicators.py** using synthetic data.
3. Call it inside **monitor/triggers.py** `evaluate()` and add a threshold check.
4. Optionally add a bull/bear vote in **monitor/assessment.py** `assess()`.
5. Expose the new value in the result dict — the UI and screener pick it up automatically.

8.2 Adding a new index to the Market Scan

1. Add the ticker list and metadata to the INDEXES dict in **monitor/universe.py**.
2. Indian NSE tickers use a **.NS** suffix; BSE use **.BO**. Other markets use Yahoo Finance's suffix convention.
3. No other changes needed — the frontend reads `index_meta()` dynamically.

8.3 Adding a new theme

Add a `key→list[str]` entry to `THEME_TICKERS` in **monitor/themes.py**. The Interests tab picks it up automatically via `GET /api/interests`.

8.4 Adding a new API endpoint

Add a route function in **webapp/main.py** following the existing pattern. FastAPI validates types automatically via Pydantic models.

Add corresponding `TestClient` tests to **tests/test_api.py** using the monkeypatched client fixture.

Out of Scope (by design)

Feature	Why excluded
Trade execution / brokerage API	Explicitly never — this tool never places trades
Price forecasting / ML predictions	Tool is alert-based; predictions would contradict all disclaimers
Multi-user / authentication	Personal tool; adding auth is straightforward with FastAPI middleware if needed
Real-time intraday data	yfinance provides daily; intraday requires a paid data feed
Hosted cloud deployment	Docker + cloud deployment can be added; current scope is local/Actions

DISCLAIMER: This document is for technical reference only. watchlist-monitor is an informational tool — it does not provide financial advice, does not forecast prices, and never places trades. Always consult a qualified financial advisor before making investment decisions.